ERS O&M Portal — API Endpoint Reference

Updated 14 October 2026. Written from `server.py` and checked against it. This replaces the version dated 2 October, which is out of date (it had no role checks, no 403 or 500 errors, and no limits on request size).

1. Start here

The server is one Python file (`server.py`) with no framework. It serves the web pages and a JSON API. There are only three kinds of call, so the whole API is small.

Kind	Pattern	Used for
Read	`GET /api/...`	Load accounts, clients, calendar events, tickets, settings, articles
Create	`POST /api/tickets` , `/api/clients` , `/api/calendar_events` , `/api/kb_articles`	Make a new record
Do something to a ticket	`POST /api/tickets/{id}/{action}`	Everything else: schedule, checklist, report, approve, close, message

There are 42 ticket actions. They all work the same way, so learn one and you know them all.

2. Rules that apply to every call

Item	Rule
Base URL	`http://localhost:8000` . The port comes from the `OM_PORT` setting.
Format	JSON in, JSON out. A POST body must be a JSON object. A list, a plain string or broken JSON is treated as an empty body `{}` .

Item	Rule
Who is calling	There is no login. Every POST carries `"actor"`, the account name, for example `"CS - Alice"`. The server checks that the name is a real account and that the account's role may do the action. It cannot prove the caller really is that person, so anyone can send another person's name. **Replace this with real login before real use.** The server must then take the actor from the login session, not from the request.
Roles	`admin` (shown as HOD), `engineer`, `cs`, `technician`.
Text clean-up	Every text value in a POST body has `<`, `>` and `"` replaced by `<`, `>` and `"` before it is stored. This stops typed text from running as code in the pages. The Knowledge Base endpoints (`/api/kb_articles...`) are the one exception, because that page escapes its own output.
Size limit	A request body over 5 MB is refused with 413 before it is read.
Times	Stored timestamps are UTC, for example `2026-10-14T09:58:07+00:00`. Visit dates are plain `YYYY-MM-DD` and "today" means today in Malaysia (UTC+8).
Live updates	There is no push. Pages ask again every 4 to 8 seconds.
One at a time	A lock makes the server handle one API request at a time. Every change rewrites the whole `db.json`.
Reading can save	Reading a ticket (one or all) first runs the automatic day check and the self-heal described in section 8. If that changes anything, `db.json` is saved. A GET is therefore not strictly read-only.
What comes back	A ticket action returns the whole updated ticket. A create returns the new record with status 201.

3. Errors

Every error has the same shape, and nothing is saved when an error is returned.

```
{"error": "A sentence the user can read."}
```

Status	Meaning	Example message
400	Wrong or missing input, or a business rule was broken. This is most errors.	`A reason is required for early closure.`
403	The account is unknown, or its role may not do this.	`Your role (Technician) can't do this. Allowed: HOD / CS.`
404	The ticket, client, article or action does not exist.	`This ticket could not be found — it may have been deleted.`
413	The request body is over 5 MB.	`That request is too large.`
500	A bug or a value of an unexpected kind. The real cause is written to `om_actions.log`.	`Something went wrong processing this action. Please try again or contact support.`

Good to know:

- Messages are plain English sentences. There are no error codes. Add a separate `code` field if the new front end needs to react to a specific error.
- A static file that does not exist returns a plain-text `Not found` with status 404, not JSON.
- Wrong-type values (a number where text is expected) return 400 `Some of the values sent were not valid. Nothing was saved.`
- Two places return 400 where 403 would be more usual: setting a technician's team, and the Knowledge Base. Treat "refused" as 400 or 403.

Order of checks on a ticket action

The server checks in this order and stops at the first problem.

1. The action name exists, otherwise 404 `Unknown action`.
2. The ticket exists, otherwise 404.
3. The actor is a real account whose role is allowed (section 4), otherwise 403.

4. For checklist actions only: the team rule (section 4), otherwise 403.
5. The ticket is not closed, otherwise 400 `This ticket is closed — it can no longer be changed.` Six actions are exempt: `message`, `mark_ticket_read`, `override_report_timestamp`, `delete_ticket`, `close` and `early_close`.
6. For report-editing actions only: the report is not submitted or approved, otherwise 400 `This report has already been submitted and can no longer be edited. Reject it to reopen for edits.`
7. The action's own rules, with its own 400 messages.

4. Who may do what

This is the table the server enforces (`ACTION_ROLES` in `server.py`). The pages hide the buttons using a copy of the same table (`ACTION_ROLES` in `static/shared.js`). **If you change one, change the other.**

What	HOD	Engineer	CS	Technician
Create a ticket, create a client	Yes	Yes	Yes	–
`assign`, `update_ticket_details`, `snooze`	Yes	Yes	Yes	–
`link_events`, `unlink_event`, `set_day_people`, `clear_reschedule`, create a calendar event	Yes	–	Yes	–
`reschedule` (ask for a new date)	–	Yes	–	Yes
The five checklist actions	–	Yes	–	Yes
Every report-editing action and `submit_report`	–	Yes	–	Yes
`report_approve`, `report_reject`	Yes	Yes	–	–
`close`, `early_close`, `delete_ticket`	Yes	–	Yes	–
`message`	Yes	Yes	Yes	– (refused by the action itself)
`mark_ticket_read`	Yes	Yes	Yes	Yes

What	HOD	Engineer	CS	Technician
`override_report_timestamp`, `POST /api/settings`	Yes	–	–	–
`POST /api/technicians/set_team`	Yes	–	Yes	–
Knowledge Base: create, update, toggle, delete	Yes	Yes	Yes	–

An engineer may approve the report that they wrote themselves. This was agreed with the HOD.

Team rule for checklists. A technician may fill a checklist only on a visit day where they are listed, or where a technician of the same team is listed. An engineer may fill one only on a visit day that has no technician and names that engineer. Otherwise the answer is 403 `You can't fill this checklist — ...`.

GET calls have no role check. Anyone who can reach the server can read every record.

5. Read endpoints

Call	Returns	Notes
`GET /api/accounts`	`{accounts: [...]}`	All 9 demo accounts. A technician has a `team`.
`GET /api/clients`	`{clients: [...]}`	
`GET /api/clients/{id}`	One client, or 404 `Client not found`	
`GET /api/calendar_events`	`{calendar_events: [...]}`	
`GET /api/kb_articles`	`{kb_articles: [...]}`	No filtering by who is asking. The Knowledge Base page decides what to show.

Call	Returns	Notes
`GET /api/tickets`	`{tickets: [...], settings: {...}}`	Every ticket in one reply, with all visit days and the full report. This will get slow as data grows. Add paging or a lighter list.
`GET /api/tickets/{id}`	One full ticket, or 404	
`GET /api/settings`	`{stale_minutes: ...}`	
any other `GET`	A file from the `static` folder	`/` opens `index.html`. A path that tries to leave the folder returns 404.

6. Create endpoints

POST /api/tickets — create a ticket

Who: HOD, Engineer, CS. Success: 201 with the new ticket in state `created`, with its report already built.

Field	Required	Notes
`client`	Yes	Must match an existing client name (capital letters do not matter). The stored spelling is used.
`type`	Yes	One of `residential_servicing`, `residential_troubleshooting`, `ci_servicing`, `ci_troubleshooting`, `site_survey`, `dere`, `ntp_phase_1`, `ntp_phase_2`, `ntp_phase_3`, `ntp_phase_4`. The old values `servicing` and `troubleshooting` are refused.
`description`	Yes	
`priority`	Yes	`low`, `medium`, `high` or `critical`.
`title`	No	If empty, becomes the client name plus the type name.
`site`	No	

Field	Required	Notes
`actor`	Yes	The creator. Becomes the first assignee.
`initial_assignees`	No	List of names. Each must be a real account and must not be a technician.
`equipment`	No	Object with `panels`, `inverter_brand`, `inverter_model`, `nemcd_ref`.
`commercial`	No	Object with `dlp_end_date` (text) and `chargeable_service_call` (true or false).
`additional`	No	Object with `so_number`, `premium_care_status`, `system_size`, `assigned_team`, `contact_phone`, `client_remarks`.

Errors: 403 wrong role. 400 for `Client, ticket type, and description are all required...`, `A valid priority level is required.`, `Please choose a valid ticket type.`, or `Client '...' is not in Clients & Sites...`.

POST /api/clients — create a client

Who: HOD, Engineer, CS. Success: 201 with the client.

Field	Required	Notes
`name`	Yes	Must be new. The check ignores capital letters.
`type`	Yes	`individual` or `company`.
`contact_person`, `phone`, `email`, `address`, `city`, `state`, `postcode`	No	Text. Not checked for format.
`actor`	Yes	Saved as `created_by`.

POST /api/calendar_events — add a calendar event

Who: HOD, CS. This stands in for the one-way Google Calendar sync, which is not built. Success: 201 with the event, `linked_ticket_id` empty.

Field	Required	Notes
`title`	Yes	
`date`	Yes	`YYYY-MM-DD`. The format is not checked.
`end_date`	No	Defaults to `date`. Must not be before `date`.
`all_day`	No	If true, `time` and `end_time` are saved empty.
`time`, `end_time`	No	`HH:MM`. On a one-day event the end time must not be before the start time.
`actor`	Yes	

POST /api/settings — change the global setting

Who: HOD. Send `stale_minutes` (a whole number). Returns the settings. 400 `stale_minutes must be a number` if it is not.

POST /api/technicians/set_team — move a technician to a team

Who: HOD or CS. Send `actor`, `name` (a technician) and `team` (`Team 1` or `Team 2`). Returns the account. All failures here are 400.

Knowledge Base articles

The Knowledge Base text is not cleaned of `<` and `>` on the way in. Whoever shows it must escape it.

Call	What it does
`POST /api/kb_articles`	Create an article. Fields: `title` (required), `category` (required: `Customer Guides`, `General Knowledge`, `Intern Resources`, `Flowcharts` or `Emergency Response`), `visibility` (`internal`, `intern_admin`, `customer` or `public`; default `internal`), `body`, `attachments` (list of file names, at most 10), `actor`. Always created as `draft`. Returns 201.

Call	What it does
`POST /api/kb_articles/{id}/update`	Change any of `title`, `category`, `visibility`, `body`, `attachments`. Only the fields you send change.
`POST /api/kb_articles/{id}/toggle_status`	Switch between `draft` and `published`.
`POST /api/kb_articles/{id}/delete`	Remove the article. Returns the article as it was.

Every call needs an `actor` who is HOD, Engineer or CS, otherwise 400 `Only admin, engineer, or CS accounts can manage Knowledge Base articles.` An unknown article is 404.

7. Ticket actions

Call: `POST /api/tickets/{id}/{action}` with a JSON body that includes `actor`. Success: 200 with the whole updated ticket. The tables use these short forms: **Send** is the body fields besides `actor`, and **Fails when** is the main 400 reasons.

7.1 People and scheduling

Action	Send	What it does	Fails when
`assign`	`name`, `action` (`add` or `remove`)	Adds or removes an assignee.	The name is not an account, or is a technician. Removing the last assignee. Removing someone who is not assigned.

Action	Send	What it does	Fails when
`link_events`	`calendar_event_ids` (list) or `calendar_event_id`; `engineer` (needed the first time); `technicians` (list)	Links calendar events to the ticket. Each date inside an event becomes its own visit day with its own empty checklist. On a ticket in `created` it moves to `scheduled`. On any other state except `closed` it is **reset to `scheduled`**.	No event chosen. Engineer missing or not an engineer. A technician name is not a technician. No technician and no engineer. Event not found. Event already linked to another ticket. Ticket closed.
`unlink_event`	`calendar_event_id`	Removes the event and its visit days. If no events are left (and the ticket is not `created` or `closed`) the ticket goes back to `created`.	Event is not linked. One of its days already has a submitted checklist.
`set_day_people`	`day_id`; `engineer` and or `technicians`	Changes who is on one visit day.	Day not found. Not an engineer or technician. A day left with nobody.

An engineer-only visit is a day with an engineer and an empty technician list. The engineer then fills the checklist alone.

7.2 Site visit and checklist

All five can only be used on the visit day itself (Malaysia date) and need `day_id`. They follow the team rule in section 4.

Action	Send	What it does	Fails when
`start_checklist`	`day_id`	Opens the day's checklist. The first time, it copies the visit date into the report's inspection date, and moves a `scheduled` ticket to `site_visit`.	Day not found. The date is in the future (`This checklist opens on ...`) or the past (`... is now past due`).
`checklist_item`	`day_id` , `item_id` , `status` (`pass` , `fail` , `na`), `note` , `attachments`	Answers one question.	Not today. Already submitted. Bad status. Item not found. A `fail` with no note and no attachment.
`checklist_general`	`day_id` , `remarks` , `media`	Saves the general remarks and photos for the day.	Not today. Already submitted.
`checklist_pdf`	`day_id` , `pdf_file`	NTP tickets only. Saves, replaces or (when empty) removes the one PDF name.	Not today. Already submitted. Day is not an NTP day.
`submit_checklist`	`day_id`	Locks the day's checklist. If every day is now submitted or unfinished, the ticket moves to `report_drafting` .	Not today. A reschedule is pending. Questions still unanswered (or no PDF for NTP).

7.3 Report editing

Who: Engineer, Technician. All of these are blocked once the report is `submitted` or `approved` . A reject opens it again. Photos are only labels (text); no file is stored.

There are two report formats. **Format A (residential)** is used by every ticket type except the two troubleshoot types. **Format B (troubleshooting)** is used by `residential_troubleshooting` and `ci_troubleshooting` . An action meant for the other

format fails with 500 (a missing field), not 400. The new front end should only offer each action for the right format.

Action	Format	Send	What it does
`report_section`	Both	`section`, `patch` (an object)	Merges `patch` into one block. Allowed `section`: `general`, `electrical`, `summary`, `report_notes`, `installation_improvement` (Format A); `checklist_info`, `intake`, `signatures`, `installation_improvement` (Format B). **The field names inside `patch` are not checked.**
`report_checklist_item`	A	`item_id`, `result` (`accept`, `reject` or null), `remarks`, `extra`	Answers one of the 16 maintenance checks.
`report_add_inverter`, `report_remove_inverter`	A	`label` (optional); `inverter_id`	Adds or removes an inverter.
`report_add_mppt`, `report_remove_mppt`	A	`inverter_id`, `label`; `inverter_id`, `mppt_id`	Adds or removes an MPPT.
`report_add_string`, `report_remove_string`, `report_update_string`	A	`inverter_id`, `mppt_id`, then `label`, or `string_id`, plus any of `voltage_v`, `pos_to_ground_v`, `neg_to_ground_v`, `status`	Manages strings.
`report_add_media`, `report_remove_media`	A and B	`category`, then `label` (add) or `index` (remove)	Categories `pv_before`, `pv_after`, `inverter`, `ac_photo`, `dc_photo`, `fit_meter` and `performance_graph` are Format A. `site_photos` is Format B. `additional` is both.

Action	Format	Send	What it does
`report_inspection_measurement`	B	`group` (`ac_three_phase`, `ac_single_phase` or `line_items`), `key`, `value`, `remark` (`ok`, `not_ok` or null)	Saves one reading.
`report_add_dc_string`, `report_remove_dc_string`, `report_update_dc_string`	B	`string_id`; any of `voltage_v`, `positive_ground_v`, `negative_ground_v` and their `..._remark`	Manages DC strings. The last one cannot be removed.
`report_narrative_add_line`, `_update_line`, `_remove_line`	B	`section` (`reported_issues`, `root_causes`, `actions_taken`, `conclusions`, `remarks`), `line_id`, `text`	Manages the free-text lines.
`report_narrative_add_photo`, `_remove_photo`	B	`section`, `line_id`, then `label` or `index`	At most 10 photos on a line.

7.4 Submit, approve, reject

Action	Who	Send	What it does	Fails when
`submit_report`	Engineer, Technician	–	Report becomes `submitted`. A ticket in `report_drafting` moves to `engineer_review`.	Already submitted. Inspection date empty (Format B: attended-site date). A visit day is still pending.
`report_approve`	HOD, Engineer	–	Report becomes `approved`. A ticket in `engineer_review` moves to `approved`.	Report is not `submitted`.

Action	Who	Send	What it does	Fails when
`report_reject`	HOD, Engineer	`reason`	Report goes back to `draft` and the reason is saved in `report_rejections`. A ticket in `engineer_review` goes back to `report_drafting`.	Report is not `submitted`. No reason.

7.5 Closing and everything else

Action	Who	Send	What it does	Fails when
`close`	HOD, CS	–	`approved` becomes `closed` (type `resolved`).	Ticket is not `approved`.
`early_close`	HOD, CS	`reason`	Any of `created`, `scheduled`, `site_visit`, `report_drafting`, `engineer_review` becomes `closed` (type `early`).	Ticket already `approved` or `closed`. No reason.
`delete_ticket`	HOD, CS	–	Removes the ticket completely.	The ticket is not in `created`.
`snooze`	HOD, Engineer, CS	`until`, `reason`	Hides the ticket until a date. The state does not change.	No date.
`reschedule`	Engineer, Technician	`reason`	Flags the ticket as needing a new date. The state does not change. While flagged, no checklist can be submitted.	No reason. Already flagged.
`clear_reschedule`	HOD, CS	–	Removes the flag after a new date is booked.	No flag is set.
`message`	HOD, Engineer, CS	`text`, `mentions` (list of names)	Adds a message. Each mention writes a line to `om_email_outbox.log`; no email is sent.	Empty text. Author is a technician. A mentioned name is not an assignable account.
`mark_ticket_read`	Everyone	–	Saves how many messages this person has seen.	–
`update_ticket_details`	HOD, Engineer, CS	`site`, `equipment`, `commercial`, `additional` (all optional)	Changes only the fields sent.	–

Action	Who	Send	What it does	Fails when
`override_report_timestamp`	HOD	`field` (`report_started_at` or `report_submitted_at`), `value` (ISO date and time), `reason`	Stores a correction beside the original time. The history is not changed.	Bad field, bad date, no reason.

8. What happens automatically

- **Day check.** Every time a ticket is read or acted on, any visit day whose date has passed without a submitted checklist is marked `unfinished`. When every day is submitted or unfinished, a `scheduled` or `site_visit` ticket moves to `report_drafting`. The server has no timer; this check stands in for a nightly job. The new system should run it on a schedule.
- **Self-heal on read.** A ticket whose report format or checklist questions do not match its type is rebuilt (only when no checklist answer exists yet). This is for old data. It can be removed once the data is clean.
- **Logs.** Every ticket action, successful or not, adds a line to `om_actions.log`. Every successful change adds an entry to the ticket's own `event_log`.
- **Daily backup.** On the first save of each Malaysian day, `db.json` is copied to `backups/db-YYYYMMDD.json`, and the newest 14 copies are kept. The copy is on the same disk, so also copy it elsewhere.
- **Safe save.** `db.json` is written to a temporary file and then swapped in, so a crash cannot leave half a file.

9. How a ticket moves

From	By	To
(new)	`POST /api/tickets`	`created`
`created`	`link_events`	`scheduled`
`scheduled`	`start_checklist`	`site_visit`
`scheduled` or `site_visit`	all days submitted or unfinished (automatic)	`report_drafting`
`report_drafting`	`submit_report`	`engineer_review`

From	By	To
`engineer_review`	`report_approve`	`approved`
`engineer_review`	`report_reject`	`report_drafting`
`approved`	`close`	`closed`
`created` to `engineer_review`	`early_close`	`closed`
`created`	`delete_ticket`	(removed)
any state except `closed`	`link_events`	`scheduled`
any state except `created` and `closed`	`unlink_event` of the last event	`created`

Watch the second-last row: linking a new event to a ticket that is already in review or approved resets it to `scheduled`, but the report keeps its old status. This is the prototype's behaviour and has not been decided with the HOD.

10. Examples

Send a message:

```
curl -X POST http://localhost:8000/api/tickets/TCK-0001/message \
  -H "Content-Type: application/json" \
  -d '{"actor":"CS - Alice","text":"Customer called, please confirm the
date."}'
```

A wrong role is refused:

```
curl -X POST http://localhost:8000/api/tickets/TCK-0001/close \
  -H "Content-Type: application/json" \
  -d '{"actor":"Technician - Ali"}'
# 403  {"error": "Your role (Technician) can't do this. Allowed: HOD / CS."}
```

11. For the digital team: what to change

1. Real login. Take `actor` from the session and ignore it in the body.
2. Return 403 consistently for permission problems (today three places use 400).
3. Add an error `code` next to the message.

4. Check the field names and types inside `report_section` `patch`; today any key is accepted.
5. Refuse a report action for the wrong format with 400 instead of failing with 500.
6. Add paging or a summary list for `GET /api/tickets`.
7. Apply role rules to GET calls (for example, the Knowledge Base `visibility` is not enforced by the server).
8. Replace the nightly-job stand-in with a real scheduled job.
9. Real file upload for photos, PDFs and article attachments (all labels today).
10. Escape text where it is shown, instead of relying on the input clean-up.